

مجموعه ۱۵ پرامپت کاربردی

نسخه کامل انگلیسی و فارسی برای Claude Code

VIBE CODING → PLAN → BUILD → TEST → SHIP

تعریف دقیق خروجی: برای هر مرحله از تعریف مسئله و طراحی تا پیاده‌سازی، تست، امنیت و نگهداری، یک پرامپت آماده و قابل شخصی‌سازی در اختیار دارید.

نکته: این پرامپت‌ها نقطه شروع حرفه‌ای هستند، اما جایگزین شناخت معماری پروژه، Code Review، Test و تصمیم مهندسی نمی‌شوند. آن‌ها را متناسب با Repository، تیم و سطح ریسک خودتان شخصی‌سازی کنید.

چطور از این مجموعه استفاده کنید؟

این فایل یک Prompt Pack آماده برای Claude Code است. هر بخش شامل نسخه اصلی انگلیسی، نسخه فارسی مستقل و توضیحی کوتاه درباره زمان و شیوه استفاده از آن است. برای نتیجه بهتر، Placeholderهای داخل کروسه را با اطلاعات واقعی پروژه جایگزین کنید و پرامپت را در Repository مرتبط اجرا کنید.

نکته مهم: قبل از اجرای هر پرامپت، Context لازم مثل Tech Stack، مسیر فایل‌ها، محدودیت‌ها و Definition of Done را دقیق وارد کنید. در کارهای حساس، تغییر روی Auth، Payment، Secret، Migration و Production Data باید با Review و تأیید انسانی انجام شود.

قاعده استفاده

- نسخه انگلیسی معمولاً برای مدل‌های کدنویسی دقیق‌تر و کم‌ابهام‌تر است.
- نسخه فارسی برای زمانی مناسب است که می‌خواهید Context و تصمیم‌ها را فارسی توضیح دهید.
- عبارت‌های داخل [] متغیر هستند و باید قبل از اجرا تکمیل شوند.
- اگر پروژه حساس است، ابتدا از Agent بخواهید Plan ارائه کند و قبل از تغییر فایل‌ها متوقف شود.
- خروجی Agent را مانند پیشنهاد یک همکار فنی بررسی کنید؛ Build، Test و Review نهایی را حذف نکنید.

۰۱ | نوشتن یک PRD کامل

این پرامپت چه کاری انجام می‌دهد؟

این پرامپت زمانی مفید است که ایده یک Feature هنوز پراکنده است و باید قبل از طراحی یا کدنویسی به یک سند تصمیم‌پذیر تبدیل شود. خروجی، مسئله، کاربران، معیارهای موفقیت، محدوده نسخه اول، تغییرات Data Model و حالت‌های شکست را مشخص می‌کند. با ذخیره PRD در مسیر ثابت، پرامپت‌های بعدی می‌توانند به همان سند رجوع کنند و احتمال تغییر ناخواسته Scope یا برداشت متفاوت از نیازها کمتر می‌شود.

نسخه انگلیسی

Write a complete PRD for the feature below.

Feature: [DESCRIBE FEATURE]

Users: [WHO USES IT]

Stack: [YOUR STACK]

Include:

- Problem statement and success metrics
- User stories with acceptance criteria
- Scope: what ships in v1 and what does not
- Data model changes
- Edge cases and failure states
- Open questions for me to answer

Keep it under 2 pages. Be specific and avoid filler.

Save it as docs/prd-[feature].md so every later prompt can reference it.

نسخه فارسی | ۰۱. نوشتن یک PRD کامل

پرامپت فارسی

برای Feature زیر یک PRD کامل بنویس.

Feature: [DESCRIBE FEATURE]

کاربران: [WHO USES IT]

Tech Stack: [YOUR STACK]

موارد زیر را پوشش بده:

- شرح دقیق مسئله و معیارهای موفقیت
- User Story ها همراه با Acceptance Criteria
- Scope: چه چیزی در ۷۱ منتشر می‌شود و چه چیزی خارج از محدوده است
- تغییرات مورد نیاز در Data Model
- Edge Case ها و Failure State ها
- سؤال‌های بازی که باید من پاسخ بدهم

سند را حداکثر در ۲ صفحه، دقیق و بدون توضیحات اضافه بنویس. آن را در مسیر docs/prd-[feature].md ذخیره کن تا پرامپت‌های بعدی بتوانند به آن ارجاع دهند.

قبل از اجرا جایگزین کنید:

[DESCRIBE FEATURE] | [WHO USES IT] | [YOUR STACK] | [feature]

۰۲ | ساخت فایل CLAUDE.md برای پروژه

این پرامپت چه کاری انجام می‌دهد؟

فایل CLAUDE.md نقش حافظه و دفترچه راهنمای پروژه را برای Claude Code دارد. این پرامپت از Agent می‌خواهد به‌جای تولید توصیه‌های عمومی، ساختار واقعی Repository، نسخه‌های مهم، Commandها، الگوهای کدنویسی و محدودیت‌های حساس را از خود Codebase استخراج کند. نتیجه باعث می‌شود Sessionهای بعدی سریع‌تر شروع شوند و Agent کمتر درباره معماری، Conventionها یا بخش‌هایی که نباید بدون اجازه تغییر کنند حدس بزند.

نسخه انگلیسی

Scan this entire codebase, then generate a CLAUDE.md.

Include:

- What this project is, in 2 lines
- Tech stack and the versions that matter
- Commands: [DEV / BUILD / TEST / LINT]
- Architecture: where things live and why
- Code conventions you actually detect in the code
- Hard rules: [YOUR NON-NEGOTIABLES] - what to never touch without asking
- Gotchas a new engineer would hit in week 1

Write rules as short imperatives. Nothing generic - only what is true for THIS repo. If you are unsure about a rule, ask me instead of inventing it.

نسخه فارسی | ۰۲. ساخت فایل CLAUDE.md برای پروژه

پرامپت فارسی

کل Codebase را بررسی کن و سپس یک فایل CLAUDE.md بساز.

موارد زیر را داخل آن بیاور:

- معرفی پروژه در حداکثر ۲ خط
- Tech Stack و Version هایی که واقعاً مهم هستند
- Command های [DEV / BUILD / TEST / LINT]
- Architecture: هر بخش کجا قرار دارد و چرا
- Code Convention هایی که واقعاً در کد تشخیص می‌دهی
- قوانین قطعی [YOUR NON-NEGOTIABLES]: چه چیزهایی را بدون اجازه من تغییر ندهی
- Gotcha هایی که یک مهندس جدید احتمالاً در هفته اول با آن‌ها روبه‌رو می‌شود

قوانین را کوتاه و امری بنویس. هیچ نکته عمومی اضافه نکن؛ فقط مواردی را بنویس که برای همین Repository درست هستند. اگر درباره قانونی مطمئن نیستی، از من سؤال کن و چیزی نساز.

قبل از اجرا جایگزین کنید:

[DEV / BUILD / TEST / LINT] | [YOUR NON-NEGOTIABLES]

۰۳ | برنامه‌ریزی عمیق پیش از کدنویسی

این پرامپت چه کاری انجام می‌دهد؟

این پرامپت برای Task‌هایی مناسب است که چند فایل یا بخش حساس را درگیر می‌کنند. Claude Code ابتدا رفتار فعلی و هدف نهایی را مقایسه می‌کند، چند Approach با Trade-off واقعی پیشنهاد می‌دهد و سپس مسیر منتخب را به گام‌های کوچک و قابل بررسی می‌شکند. توقف اجباری پیش از هر تغییر، فرصت می‌دهد Plan را تأیید کنید و برای بخش‌هایی مثل Auth، Payment یا Production Data کنترل انسانی داشته باشید.

نسخه انگلیسی

Enter plan mode. Do NOT write any code yet.

Task: [PASTE TASK]

Constraints: [DEADLINE / STACK / NO-GO ZONES]

1. Read every file this task touches. List each file with one line explaining what it does today.
2. Map current behavior versus target behavior.
3. Propose 2-3 approaches with real trade-offs: complexity, risk, and blast radius.
4. Pick one approach and justify it in 3 lines.
5. Break it into steps small enough to verify one at a time, each with its own check.
6. List the risks and the exact rollback for each one.
7. Flag anything touching [AUTH / PAYMENTS / PROD DATA] for my explicit sign-off.

Then stop. Show me the plan and wait for my approval before touching a single file.

نسخه فارسی | ۰۳. برنامه‌ریزی عمیق پیش از کدنویسی

پرامپت فارسی

وارد Plan Mode شو و هنوز هیچ کدی ننویس.

Task: [PASTE TASK]

محدودیت‌ها: [DEADLINE / STACK / NO-GO ZONES]

۱. همه فایل‌هایی را که این Task با آن‌ها درگیر می‌شود بخوان و برای هر فایل در یک خط توضیح بده اکنون چه کاری انجام می‌دهد.
 ۲. رفتار فعلی را با رفتار هدف مقایسه کن.
 ۳. دو تا سه Approach با Trade-off واقعی پیشنهاد بده و Risk، Complexity، و Blast Radius هرکدام را توضیح بده.
 ۴. یک Approach را انتخاب کن و دلیل انتخاب را در ۳ خط بنویس.
 ۵. مسیر اجرا را به گام‌هایی تقسیم کن که بتوان هرکدام را جداگانه بررسی کرد؛ برای هر گام یک Check مشخص بنویس.
 ۶. Riskها و Rollback دقیق هرکدام را فهرست کن.
 ۷. هر مورد مرتبط با [AUTH / PAYMENTS / PROD DATA] را برای تأیید صریح من علامت‌گذاری کن.
- سپس متوقف شو، Plan را نمایش بده و قبل از تغییر حتی یک فایل منتظر تأیید من بمان.

قبل از اجرا جایگزین کنید:

[PASTE TASK] | [DEADLINE / STACK / NO-GO ZONES] | [AUTH / PAYMENTS / PROD DATA]

۴. | توسعه مبتنی بر Spec

این پرامپت چه کاری انجام می‌دهد؟

در این روش ابتدا رفتار مورد انتظار به یک Spec قابل آزمون تبدیل می‌شود و کد فقط بعد از تأیید آن نوشته خواهد شد. پرامپت، Happy Path، Edge Case، Failure State، قرارداد API، تغییرات Schema و حالت‌های UI را روشن می‌کند. اگر هنگام پیاده‌سازی واقعیت پروژه با Spec ناسازگار باشد، Agent باید توقف کند و تغییر را ابتدا در سند ثبت کند؛ بنابراین منبع تصمیم‌گیری از کد پراکنده به یک سند قابل بررسی منتقل می‌شود.

نسخه انگلیسی

We build specs first. Write a spec for: [FEATURE]

Context: [WHO USES IT + WHY NOW]

Spec format:

- Behavior: given / when / then for every case - happy path, edge cases, and failure states
- API contract: inputs, outputs, error shapes, and status codes
- Data: schema changes and migrations needed
- UI states: loading, empty, error, and success
- Non-goals: what this spec deliberately skips
- An acceptance checklist I can verify line by line

After I approve the spec, implement EXACTLY the spec. If reality forces a deviation, stop, update the spec, get my approval, and then continue. The spec is the source of truth, not the code.

نسخه فارسی | ۰۴. توسعه مبتنی بر Spec

پرامپت فارسی

ما ابتدا Spec می‌نویسیم. برای Feature زیر یک Spec کامل آماده کن:

Feature: [FEATURE]

Context: [WHO USES IT + WHY NOW]

ساختار Spec:

- Behavior: همه سناریوها را با قالب given / when / then بنویس؛ شامل Happy Path، Edge Case و Failure State

- API Contract: Input ها، Output ها، ساختار Error و Status Code ها

- Data: تغییرات Schema و Migration های موردنیاز

- UI State ها: Success و Loading، Empty، Error

- Non-goal ها: چه مواردی عمداً در این Spec نیستند

- Acceptance Checklist که بتوانم خطبه خط بررسی کنم

بعد از تأیید من، دقیقاً مطابق Spec پیاده‌سازی کن. اگر واقعیت پروژه نیازمند تغییر بود، متوقف شو، ابتدا Spec را به‌روزرسانی کن، تأیید من را بگیر و سپس ادامه بده. منبع اصلی حقیقت Spec است، نه کد.

قبل از اجرا جایگزین کنید:

[FEATURE] | [WHO USES IT + WHY NOW]

۵۰ | ساخت Design Brief کامل برای UI و UX

این پرامپت چه کاری انجام می‌دهد؟

این پرامپت قبل از طراحی Screen یا Flow، تصمیم‌های تجربه کاربری و سیستم بصری را یکجا مشخص می‌کند. خروجی فقط یک Wireframe کلی نیست؛ مسیر کاربر، Component State، Spacing، Breakpoint، Hierarchyها، Typography، Color Token، Motion و Accessibility را پوشش می‌دهد. اشاره به محصولات مرجع برای جهت‌گیری است و دستور «کپی نکن» کمک می‌کند نتیجه از الگوها یاد بگیرد، اما هویت مستقل محصول حفظ شود.

نسخه انگلیسی

Create a full UI/UX design brief for: [SCREEN OR FLOW]

Audience: [USERS]

Brand: [COLORS / FONTS / VIBE]

Deliver:

- User journey through the flow, step by step
- Layout per screen: hierarchy, spacing, and breakpoints
- Component inventory with every state: hover, empty, error, and loading
- Typography and color tokens
- Motion: what animates, duration, and easing
- Accessibility notes

Study patterns from [2-3 PRODUCTS YOU ADMIRE] for direction. Never copy them.

نسخه فارسی | ۰.۵ ساخت Design Brief کامل برای UI و UX

پرامپت فارسی

برای Screen یا Flow زیر یک UI/UX Design Brief کامل بساز:

Flow: [SCREEN OR FLOW] یا Screen

مخاطب: [USERS]

Brand: [COLORS / FONTS / VIBE]

خروجی باید شامل این موارد باشد:

- User Journey مرحله به مرحله در کل Flow

- Layout هر Screen شامل Hierarchy، Spacing و Breakpoint ها

- فهرست Component ها و تمام State های آن ها: Loading و Hover، Empty، Error

- Typography و Color Token ها

- Motion: چه چیزی Animate می شود، Duration چقدر است و از چه Easing استفاده می شود

- نکات Accessibility

برای جهت گیری، Pattern های [2-3 PRODUCTS YOU ADMIRE] را بررسی کن؛ اما هیچ کدام را کپی نکن.

قبل از اجرا جایگزین کنید:

[SCREEN OR FLOW] | [USERS] | [COLORS / FONTS / VIBE] | [2-3 PRODUCTS YOU ADMIRE]

۰۶ | ساخت Implementation Plan مرحله‌ای

این پرامپت چه کاری انجام می‌دهد؟

پس از تأیید PRD یا Spec، این پرامپت آن را به یک Build Sequence کنترل‌شده تبدیل می‌کند. هر مرحله باید کوچک، قابل Verify و از نظر اثر روی Build مستقل باشد؛ یعنی پروژه بعد از هر گام همچنان Compile و Run شود. مشخص کردن File‌های درگیر، Risk، Dependency، Migration و اندازه S/M/L کمک می‌کند اجرای کار قابل تخمین، توقف و Rollback باشد و Unknown‌های پرریسک زودتر آشکار شوند.

نسخه انگلیسی - آماده کپی

Create an implementation plan for: [APPROVED SPEC / PRD]

Rules:

- Sequence the steps so the app compiles and runs after EVERY single step.
- For each step, list the files touched, what changes, and how I verify it works.
- Flag steps that need a migration or a new dependency.
- Put the riskiest unknowns first.
- Size each step: [S / M / L]

Output a numbered build sequence I can run one step at a time. Wait for my "go" between steps.

نسخه فارسی | ۰۶. ساخت Implementation Plan مرحله‌ای

پرامپت فارسی

برای سند تأییدشده زیر یک Implementation Plan بساز:

سند ورودی: [APPROVED SPEC / PRD]

قوانین:

- مراحل را طوری مرتب کن که Application بعد از هر مرحله Compile و Run شود.
- برای هر مرحله، File‌های درگیر، تغییر مورد انتظار و روش Verify را مشخص کن.
- مرحله‌هایی را که به Migration یا Dependency جدید نیاز دارند علامت‌گذاری کن.
- Unknown‌های پریسک را زودتر بررسی کن.
- اندازه هر مرحله را با [S / M / L] مشخص کن.

یک Build Sequence شماره‌گذاری‌شده بده که بتوانم هر بار فقط یک مرحله را اجرا کنم. بین مراحل منتظر پیام go من بمان.

قبل از اجرا جایگزین کنید:

[APPROVED SPEC / PRD] | [S / M / L]

۰۷ | اتصال یک MCP Server به پروژه

این پرامپت چه کاری انجام می‌دهد؟

این پرامپت اتصال یک Service یا API به Claude Code از طریق MCP را مرحله‌بندی می‌کند. ابتدا باید وجود Server رسمی یا معتبر بررسی شود؛ در غیر این صورت Agent یک Server حداقلی با MCP SDK می‌سازد. محدود کردن Toolها، نگهداری Secretها در Environment Variable، تست End-to-End و مستندسازی یک خطی هر Tool باعث می‌شود اتصال قابل نگهداری، امن‌تر و برای Sessionهای آینده قابل کشف باشد.

نسخه انگلیسی

Wire up an MCP server for: [SERVICE / API]

What I need it to do: [JOBS TO BE DONE]

1. Check for an official or well-maintained existing server first - name your source.
2. If one exists, provide the exact install command and the .mcp.json config, scoped to this project.
3. If not, scaffold one with the MCP SDK, including tools, auth, error handling, and typed responses.
4. Add ONLY the tools I will actually use: [LIST]
5. Wire secrets through [ENV VARS]. Never hardcode keys in the config file.
6. Verify the connection and call one tool end to end. Show me the output.
7. Document each tool in 1 line so future sessions know when to use it.

نسخه فارسی | ۰۷. اتصال یک MCP Server به پروژه

پرامپت فارسی - آماده کپی

برای Service یا API زیر یک MCP Server متصل کن:

Service یا [SERVICE / API] API:

کاری که باید انجام دهد: [JOBS TO BE DONE]

۱. ابتدا بررسی کن آیا Server رسمی یا معتبر و فعال وجود دارد؛ Source را نام ببر.
۲. اگر وجود دارد، Command دقیق نصب و Config فایل mcp.json را در Scope همین پروژه ارائه کن.
۳. اگر وجود ندارد، با MCP SDK یک Server بساز که Toolها، Auth، Error Handling و Typed Response داشته باشد.
۴. فقط Toolهایی را اضافه کن که واقعاً استفاده می‌کنم: [LIST]
۵. Secretها را از طریق [ENV VARS] متصل کن و هیچ Key را داخل Config هاردکد نکن.
۶. Connection را Verify کن، یک Tool را End-to-End اجرا کن و Output را نشان بده.
۷. کاربرد هر Tool را در یک خط مستند کن تا Sessionهای بعدی بدانند چه زمانی از آن استفاده کنند.

قبل از اجرا جایگزین کنید:

[SERVICE / API] | [JOBS TO BE DONE] | [LIST] | [ENV VARS]

۰۸ | اتصال اصولی Database

این پرامپت چه کاری انجام می‌دهد؟

این پرامپت اتصال Database را از انتخاب Client تا اثبات خواندن و نوشتن داده پوشش می‌دهد. تمرکز آن فقط روی برقرارشدن Connection نیست؛ Schema، Relation، Index، Migration و Typed Query، Rollback، Helper، RLS برای Multi-tenant و Connection Pooling در Serverless را نیز مشخص می‌کند. تست Seed و Read-back در پایان نشان می‌دهد مسیر واقعی داده کار می‌کند و تنظیمات فقط روی کاغذ باقی نمانده‌اند.

نسخه انگلیسی

Connect this app to [POSTGRES / SUPABASE / YOUR DB].

- Pick the client that fits this stack and justify it in 1 line.
- Env vars: name them, add them to .env.example, and never commit real values.
- Schema: create tables for [ENTITIES] with types, relations, and indexes for [HOT QUERIES].
- Create and run the migrations, and show the rollback for each one.
- Add one typed query helper per table - no raw SQL scattered through components.
- Add access rules or row-level security if this is [MULTI-TENANT].
- Add connection pooling if we are serverless.

Then prove it: seed one row, read it back, and show me the output.

نسخه فارسی | ۰.۸. اتصال اصولی Database

پرامپت فارسی

این Application را به Database زیر متصل کن:

Database: [POSTGRES / SUPABASE / YOUR DB]

- Client متناسب با Stack را انتخاب کن و دلیل آن را در یک خط بنویس.
 - نام Env Var ها را مشخص کن، آن‌ها را به env.example. اضافه کن و مقدار واقعی را هرگز Commit نکن.
 - برای Table [ENTITIES] هایی با Type، Relation و Index مناسب برای [HOT QUERIES] طراحی کن.
 - Migration ها را بساز و اجرا کن و Rollback هرکدام را نشان بده.
 - برای هر Table یک Typed Query Helper بساز و Raw SQL را در Component ها پخش نکن.
 - اگر سیستم [MULTI-TENANT] است، Access Rule یا Row-Level Security اضافه کن.
 - اگر محیط Serverless است، Connection Pooling را در نظر بگیر.
- در پایان اثبات کن اتصال کار می‌کند: یک Row را Seed کن، دوباره آن را بخوان و Output را نشان بده.

قبل از اجرا جایگزین کنید:

[POSTGRES / SUPABASE / YOUR DB] | [ENTITIES] | [HOT QUERIES] | [MULTI-TENANT]

۰۹ | پیدا کردن شکاف‌های امنیتی

این پرامپت چه کاری انجام می‌دهد؟

این پرامپت یک Security Audit عملی روی Codebase اجرا می‌کند و یافته‌ها را از یک فهرست عمومی به موارد قابل اقدام تبدیل می‌کند. بررسی CVE Dependency، Upload، Validation، IDOR، Auth، Injection، Secret، Rate Limit و نشت اطلاعات انجام می‌شود. الزام به File:Line، Severity و Effort Estimate باعث می‌شود هر Finding قابل پیگیری باشد. اصلاح خودکار باید فقط در Scope مجاز انجام شود و تغییرات حساس همچنان نیازمند Review انسانی هستند.

نسخه انگلیسی

Audit this codebase for security gaps. Attack it like you want in.

Focus areas: [AUTH / PAYMENTS / USER DATA]

Check:

- Secrets in code, config, or git history
- Injection: SQL, XSS, command, and path traversal
- Auth: routes missing checks, weak sessions, and broken redirects
- IDOR: can user A read user B's data?
- File uploads and input validation on every form
- Dependency CVEs - run the audit and read the output
- Rate limiting on [EXPENSIVE ENDPOINTS]
- What leaks through error messages and logs

Rank findings by severity with exact file:line references. Fix the critical findings now, and list the rest as tickets with effort estimates.

نسخه فارسی | ۰۹. پیدا کردن شکاف‌های امنیتی

پرامپت فارسی

این Codebase را از نظر Security Gap بررسی کن و از زاویه یک مهاجم به آن نگاه کن.

حوزه‌های تمرکز: [AUTH / PAYMENTS / USER DATA]

موارد زیر را بررسی کن:

- Secret های موجود در Code، Config یا Git History
- Injection شامل SQL، XSS، Command و Path Traversal
- Auth شامل Route های بدون Session، Check، Redirect و آسیب‌پذیر
- IDOR: آیا User A می‌تواند Data مربوط به User B را بخواند؟
- File Upload و Input Validation در تمام Form ها
- CVE Dependency ها؛ Audit را اجرا کن و Output را تحلیل کن
- Rate Limiting روی [EXPENSIVE ENDPOINTS]
- اطلاعاتی که از Error Message ها و Log ها نشت می‌کنند

Finding ها را همراه با File:Line دقیق و بر اساس Severity رتبه‌بندی کن. موارد Critical را در Scope مجاز اصلاح کن و بقیه را به شکل Ticket همراه با Effort Estimate فهرست کن.

قبل از اجرا جایگزین کنید:

[AUTH / PAYMENTS / USER DATA] | [EXPENSIVE ENDPOINTS]

۱۰ | رفع سریع خطا بر اساس شواهد

این پرامپت چه کاری انجام می‌دهد؟

این پرامپت جلوی Debugging مبتنی بر حدس را می‌گیرد. ابتدا Stack Trace و فایل‌های درگیر خوانده می‌شوند، سپس Expected Behavior و Actual Behavior روشن می‌شود و Hypothesis ها با Log یا Test کوچک سنجیده خواهند شد. پس از یافتن Root Cause، الگوی مشابه در Repository جست‌وجو و Regression Test اضافه می‌شود. نتیجه فقط رفع علامت ظاهری نیست؛ دلیل شکست و حفاظی که مانع تکرار آن می‌شود نیز مستند خواهد شد.

نسخه انگلیسی

Debug this error. Do NOT guess.

Error: [PASTE FULL ERROR + STACK TRACE]

When it happens: [STEPS TO REPRODUCE]

1. Read the stack trace and open the exact files involved.
2. State expected versus actual behavior in 1 line.
3. List 3 hypotheses, ranked by likelihood.
4. Prove or eliminate each one with logs or a tiny test - evidence, not vibes.
5. Fix the root cause, not the symptom.
6. Search the repo for the same pattern. If it can break here, it may break elsewhere.
7. Add a regression test that fails without the fix.
8. Tell me in 2 lines why it broke and why it cannot break this way again.

نسخه فارسی | ۱۰. رفع سریع خطا بر اساس شواهد

پرامپت فارسی

این Error را Debug کن و حدس نزن.

Error: [PASTE FULL ERROR + STACK TRACE]

زمان وقوع: [STEPS TO REPRODUCE]

۱. Stack Trace را بخوان و File های دقیق درگیر را باز کن.
۲. Expected Behavior و Actual Behavior را در یک خط بیان کن.
۳. سه Hypothesis را بر اساس احتمال مرتب کن.
۴. هر Hypothesis را با Log یا یک Test کوچک تأیید یا رد کن؛ فقط بر اساس Evidence پیش برو.
۵. Root Cause را اصلاح کن، نه Symptom را.
۶. Repository را برای Pattern مشابه جست و جو کن؛ اگر اینجا می شکند ممکن است جای دیگری هم بشکند.
۷. یک Regression Test اضافه کن که بدون Fix شکست بخورد.
۸. در ۲ خط توضیح بده چرا خطا رخ داد و چه چیزی مانع تکرار آن می شود.

قبل از اجرا جایگزین کنید:

[PASTE FULL ERROR + STACK TRACE] | [STEPS TO REPRODUCE]

۱۱ | نوشتن تست‌های E2E با Playwright

این پرامپت چه کاری انجام می‌دهد؟

این پرامپت برای پوشش مسیری‌هایی است که مستقیماً از نگاه کاربر اهمیت دارند. ابتدا Money Path یا Core Action تست می‌شود و Selectorها بر اساس Role و Label انتخاب می‌شوند تا Test به CSS شکننده وابسته نباشد. مستقل بودن Testها، Seed جداگانه Data، اجرای Headless در CI و ذخیره Screenshot و Trace فقط هنگام Failure، Suite را قابل اعتمادتر و Debug آن را سریع‌تر می‌کند.

نسخه انگلیسی

Write Playwright E2E tests for: [FLOW]

Stack: [YOUR STACK]

CI: [GITHUB ACTIONS / OTHER]

- Test money paths first: [SIGNUP / CHECKOUT / CORE ACTION]
- Test what the user sees, not implementation details.
- Use roles and labels as selectors, never brittle CSS chains.
- Add one unhappy path per flow: bad input, network failure, or expired session.
- Keep tests independent: any order, zero shared state, and each test seeds its own data.
- Run headless in CI and headed locally for debugging.
- Capture screenshots and traces on failure only.

Run the suite, show me the results, fix what fails, and tell me what the suite still does NOT cover.

نسخه فارسی | ۱۱. نوشتن تست‌های EYE با Playwright

پرامپت فارسی - آماده کپی

برای Flow زیر با Playwright تست EYE بنویس:

Flow: [FLOW]

Stack: [YOUR STACK]

CI: [GITHUB ACTIONS / OTHER]

- ابتدا Money Path ها را تست کن: [SIGNUP / CHECKOUT / CORE ACTION]
- چیزی را تست کن که User می‌بیند، نه جزئیات Implementation را.
- برای Selector از Role و Label استفاده کن و به CSS Chain شکننده وابسته نشو.
- برای هر Flow یک Unhappy Path مانند Input نامعتبر، Network Failure یا Session منقضی اضافه کن.
- Test ها باید مستقل باشند، با هر ترتیبی اجرا شوند، Shared State نداشته باشند و Data خود را Seed کنند.
- در CI به صورت Headless و برای Debug محلی به صورت Headed اجرا کن.
- Screenshot و Trace را فقط هنگام Failure ذخیره کن.
- Suite را اجرا کن، نتیجه را نشان بده، Failure ها را اصلاح کن و مشخص کن Suite هنوز چه چیزهایی را پوشش نمی‌دهد.

قبل از اجرا جایگزین کنید:

[FLOW] | [YOUR STACK] | [GITHUB ACTIONS / OTHER] | [SIGNUP / CHECKOUT / CORE ACTION]

۱۲ | پاک‌سازی Dead Code با اطمینان

این پرامپت چه کاری انجام می‌دهد؟

حذف Dead Code می‌تواند ساده به نظر برسد، اما Reference، Dynamic Import، متنی و Feature Flag ها خطر حذف اشتباه را بالا می‌برند. این پرامپت Agent را مجبور می‌کند پیش از هر Delete جست‌وجو انجام دهد، تغییرات را کوچک نگه دارد و بعد از هر مرحله Build و Test را اجرا کند. گزارش تعداد Line های حذف‌شده و موارد مشکوک نیز مرز میان حذف مطمئن و بخش‌هایی را که به تصمیم انسان نیاز دارند روشن می‌کند.

نسخه انگلیسی - آماده کپی

Find and delete dead code in this repo.

Scope: [WHOLE REPO / SPECIFIC FOLDER]

- Unused exports, components, hooks, and utilities
- Unreachable branches and commented-out blocks
- package.json dependencies that nothing imports
- Stale feature flags stuck always-on or always-off
- Duplicate logic that should merge into one implementation
- Dead CSS classes and unused assets

Verify with a search before EVERY deletion - dynamic imports and string references count.

Delete in small commits, run [BUILD + TESTS] after each one, and report the total lines removed plus anything you were not 100% sure about.

نسخه فارسی | ۱۲. پاک‌سازی Dead Code با اطمینان

پرامپت فارسی - آماده کپی

Dead Code های این Repository را پیدا و حذف کن.

Scope: [WHOLE REPO / SPECIFIC FOLDER]

- Utility و Export، Component، Hook بدون استفاده
- Branch های غیرقابل دسترسی و Block های Comment شده
- Dependency های package.json که هیچ‌جا Import نشده‌اند
- Feature Flag های قدیمی که همیشه On یا همیشه Off مانده‌اند
- Logic تکراری که باید در یک Implementation ادغام شود
- CSS Class های مرده و Asset های بدون استفاده

قبل از هر Delete با Search مطمئن شو چیزی به آن Reference ندارد؛ Dynamic Import و String Reference را هم حساب کن. حذف‌ها را در Commit های کوچک انجام بده، بعد از هرکدام [BUILD + TESTS] را اجرا کن و در پایان تعداد کل Line های حذف‌شده و هر موردی را که درباره آن ۱۰۰٪ مطمئن نبودی گزارش بده.

قبل از اجرا جایگزین کنید:

[WHOLE REPO / SPECIFIC FOLDER] | [BUILD + TESTS]

۱۳ | نوشتن Git Commit های تمیز

این پرامپت چه کاری انجام می‌دهد؟

این پرامپت تغییرات Stage شده را به Commit های مستقل و قابل فهم تقسیم می‌کند. پیام‌ها بر اساس Convention پروژه نوشته می‌شوند و Subject کوتاه، امری و متمرکز بر «چه چیزی و چرا» خواهد بود. جداکردن Refactor از Behavior Change، حذف Secret و Generated File از Commit و نمایش Plan پیش از اجرا، History پروژه را برای Review، Rollback و Debug آینده بسیار ارزشمندتر می‌کند.

نسخه انگلیسی

Commit my staged changes properly.

Convention: [CONVENTIONAL COMMITS / YOUR FORMAT]

- Split unrelated changes into separate commits.
- Format: type(scope): what changed and why
feat / fix / refactor / chore / docs / test
- Keep the subject under 50 characters and use imperative mood.
- Explain WHY in the body, wrapped at 72 characters.
- Reference the ticket: [TICKET-ID]
- Never mix a refactor with a behavior change in one commit.
- Never commit [SECRETS / .ENV / GENERATED FILES]

Show me the plan - files per commit and proposed messages - before you commit anything. Then commit one at a time so I can stop you between them.

نسخه فارسی | ۱۳. نوشتن Git Commit های تمیز

پرامپت فارسی - آماده کپی

تغییرات Stage شده را به شکل اصولی Commit کن.

Convention: [CONVENTIONAL COMMITS / YOUR FORMAT]

- تغییرات نامرتبط را در Commit های جدا قرار بده.
- Format پیام: type(scope): what changed and why
- Type ها: feat / fix / refactor / chore / docs / test
- Subject کمتر از ۵۰ Character و با لحن امری باشد.
- در Body دلیل تغییر را توضیح بده و Line ها را در ۷۲ Character Wrap کن.
- به Ticket ارجاع بده: [TICKET-ID]
- Refactor و Behavior Change را در یک Commit ترکیب نکن.
- موارد [SECRETS / .ENV / GENERATED FILES] را Commit نکن.

قبل از هر Plan، Commit، شامل File های هر Commit و Message پیشنهادی را نشان بده. سپس Commit ها را یکی یکی انجام بده تا بتوانم بین آنها متوقف کنم.

قبل از اجرا جایگزین کنید:

[CONVENTIONAL COMMITS / YOUR FORMAT] | [TICKET-ID] | [SECRETS / .ENV /
GENERATED FILES]

۱۴ | استفاده از Hook به عنوان Guardrail

این پرامپت چه کاری انجام می‌دهد؟

Hook ها کنترل‌های تکرارشونده را از Prompt به سطح اجرای Agent منتقل می‌کنند. با PostToolUse می‌توان Lint و Typecheck را بعد از Edit اجرا کرد، با PreToolUse مسیرهای حساس را بست، با Stop تست نهایی گرفت و با Notification نیاز به ورودی انسان را اعلام کرد. کوتاه‌بودن Script ها و Error Message روشن مهم است؛ چون Agent باید دقیقاً بفهمد چه چیزی شکست خورده و برای ادامه چه اقدامی لازم است.

نسخه انگلیسی

Set up Claude Code hooks as guardrails here.

Stack: [YOUR STACK + PACKAGE MANAGER]

- PostToolUse: run [LINT + TYPECHECK] after every file edit and feed errors straight back.
- PreToolUse: block edits to [PROTECTED PATHS: MIGRATIONS, .ENV, PROD CONFIG].
- Stop: run [TEST SUITE] before a session ends.
- Notification: ping me with [SOUND / SLACK] when my input is needed.

Write the hook scripts and the settings.json entries. Keep each script under 20 lines and exit non-zero with a clear message so the agent knows exactly what to fix. Then trigger each hook on purpose and show me it firing.

نسخه فارسی | ۱۴. استفاده از Hook به عنوان Guardrail

پرامپت فارسی - آماده کپی

در این پروژه Hook های Claude Code را به عنوان Guardrail تنظیم کن.

Stack: [YOUR STACK + PACKAGE MANAGER]

- PostToolUse: بعد از هر [LINT + TYPECHECK]، File Edit، را اجرا کن و Error ها را مستقیم برگردان.
- PreToolUse: Edit روی [PROTECTED PATHS: MIGRATIONS, .ENV, PROD CONFIG] را مسدود کن.

- Stop: قبل از پایان [TEST SUITE]، Session، را اجرا کن.
- Notification: وقتی به Input من نیاز است از طریق [SOUND / SLACK] اطلاع بده.

Script Hook ها و Entry های فایل settings.json را بنویس. هر Script کمتر از ۲۰ Line باشد و در صورت خطا با Exit Code غیر صفر و Message واضح متوقف شود تا Agent دقیقاً بداند چه چیزی را باید اصلاح کند. سپس هر Hook را عمداً Trigger کن و اجرای آن را نشان بده.

قبل از اجرا جایگزین کنید:

[YOUR STACK + PACKAGE MANAGER] | [LINT + TYPECHECK] | [PROTECTED PATHS:
MIGRATIONS, .ENV, PROD CONFIG] | [TEST SUITE] | [SOUND / SLACK]

۱۵ | تبدیل یک کار تکراری به Skill

این پرامپت چه کاری انجام می‌دهد؟

وقتی یک Workflow را بارها با Prompt مشابه اجرا می‌کنید، تبدیل آن به Skill باعث می‌شود Trigger، مراحل، Convention ها و تعریف Done همیشه ثابت بمانند. این پرامپت از Claude Code می‌خواهد Skill را با Frontmatter دقیق، Workflow شماره‌گذاری‌شده، Edge Case و مرز میان سؤال کردن و استنباط کردن بسازد. Dry-run روی یک مثال واقعی نیز نشان می‌دهد Skill واقعاً مانند روش دستی شما عمل می‌کند یا هنوز نیاز به اصلاح دارد.

نسخه انگلیسی

Turn this repetitive task into a Claude Code skill.

The task I keep doing: [DESCRIBE TASK + STEPS]

- Create .claude/skills/[NAME]/SKILL.md
- Frontmatter: name and description with the exact trigger phrases I actually say
- Body: numbered workflow, my conventions, and edge cases
- What it should ask me for versus infer on its own
- What "done" looks like

Then dry-run the skill on a real example and refine it until the output matches how I do it by hand.

نسخه فارسی | ۱۵. تبدیل یک کار تکراری به Skill

پرامپت فارسی - آماده کپی

این Task تکراری را به یک Skill برای Claude Code تبدیل کن.

وظیفه ای که مرتب انجام می‌دهم: [DESCRIBE TASK + STEPS]

- فایل `claude/skills/[NAME]/SKILL.md` را بساز.

- در `Frontmatter`، `Name` و `Description` را همراه با `Trigger Phrase` های دقیقی بنویس که من واقعاً استفاده می‌کنم.

- در `Body`، `Workflow`، `شماره‌گذاری‌شده`، `Convention` های من و `Edge Case` ها را ثبت کن.

- مشخص کن Skill چه اطلاعاتی را باید از من بپرسد و چه چیزهایی را می‌تواند خودش Infer کند.

- تعریف دقیق `Done` را بنویس.

سپس Skill را روی یک مثال واقعی `Dry-run` کن و آن‌قدر اصلاحش کن تا `Output` با روشی که من به‌صورت دستی انجام می‌دهم هماهنگ شود.

قبل از اجرا جایگزین کنید:

[DESCRIBE TASK + STEPS] | [NAME]

چک‌لیست قبل از اجرای پرامپت

- Placeholderها را با اطلاعات واقعی پروژه جایگزین کرده‌ام.
- Scope و بخش‌هایی که نباید تغییر کنند مشخص هستند.
- مسیر Build، Test، Lint و Typecheck را به Agent داده‌ام.
- برای تغییرات حساس، مرحله تأیید انسانی تعریف کرده‌ام.
- از Agent خواسته‌ام نتیجه، Evidence و روش Verify را گزارش کند.
- پیش از Merge یا Deploy، خروجی را Review و Test می‌کنم.

برای یادگیری ساخت اتوماسیون‌ها و ایجنت‌های هوش مصنوعی با ما همراه باشید؛ و اگر می‌خواهید طراحی، پیاده‌سازی و استقرار راهکارهای هوش مصنوعی کسب‌وکاران را به یک تیم متخصص بسپارید، با ما در ارتباط باشید.

Follow us for AI insights, training, collaboration opportunities, and custom AI solutions.

Instagram: [@madadi_ai](#) | [@nexgent_ai](#)

LinkedIn: [Mostafa Madadi](#)

Email: nexgent.ai.group@gmail.com